

Inigo Quilez :: articles :: rays and polygons - 2010



This is basic stuff, but why not to refresh memory from time to time and revisit the basic concepts once again. Say you are raymarching or raytracing some objects in a fragment shader, and you want to composite them with some other geometry that you rendered or will render through regular rasterization. The only thing you need to do is to output a depth value in your raytracing/marching shader, and let the depth buffer do the rest. The first to do, then, is to understand what "depth" means here.

In a raytracer/marcher, you probably have access to the distance from the ray origin (you camera position) to the closest geometry/intersection point. That distance is **not** what you want to write to the depth buffer, as hardware rasterizers (OpenGL or DirectX) don't store distances to the camera, but the z of the geometry/intersection point. The reason is that this z value is still monotonically increasing with the distance, but has the property of being linear (linear like in "can be interpolated across the surface of a play 3D triangle). So, in your raymarcher, compute the intersection point, and use its z component for writing to the depth buffer.

Well, that will not work just like that. Your rendering API will remap your z values to a -1 to 1 range (or to the 0 to 1 range) based on the near and far clipping planes you decided to set up. Furthermore, the remapping will probably also transform your z values to some other sort of scale that exploits the properties of perspective (like with a curve that compresses values in the far distance). So you will have to implement the same remapping in your shader before you can merge your raytraced/marched objects with the rest of the polygons.

The mapping is simple, though, and is normally configured by the projection matrix. Grab your OpenGL Redbook, and have a look to the content of a standard projection matrix. The third and fourth row are what we need, since those are the ones that affect the z and w components of your points when transformed from eye to clip space. So, if ze is the z of your intersection point in camera (eye) space, then you can compute the clip space z and w as

$$zc = -ze * (far + near) / (far - near) - 2 * far * near / (far - near)$$

$$wc = -ze$$

The hardware will then do the perspective division and compute the z value in normalized device coordinates before converting it to a 24 bit depth value:

$$z_n = z_c/z_w = (\text{far}+\text{near})/(\text{far}-\text{near}) + 2*\text{far}*\text{near}/(\text{far}-\text{near})/z_e$$

which you can see it is a formula of the form $z_n = a + b/z_e$ which produces the desired depth compression.

You can check that the boundary conditions are met, by doing

$$z_e = -\text{near} \rightarrow z_n = -1;$$

$$z_e = -\text{far} \rightarrow z_n = 1;$$

Yeah, remember that your depths in camera space are negative inwards the screen. So, our raytracing/marching shader should end with something like

```
float a = (far+near)/(far-near);  
float b = 2.0*far*near/(far-near);  
gl_FragDepth = a + b/z;
```

You probably want to upload a and b as uniforms to your shader.

Alternatively, if you don't want mess with all this, you can directly grab the projection parameters from the projection matrix, and do something like

```
float zc = ( (ModelView)ProjectionMatrix * vec4(intersectionPoint,1.0) ).z;  
float wc = ( (ModelView)ProjectionMatrix * vec4(intersectionPoint,1.0) ).w;  
gl_FragDepth = zc/wc;
```

which is a little bit more expensive, but gives the same results...

inigo quilez - learning computer graphics since 1994